

EveryFinance/smart-contracts- Stellar@0a6773be0f4919d3cc542d56cb31977d9735d965

Status: COMPLETE
Report Generated: 5/9/2026, 6:16:38 AM
Scan Date: May 9, 2026, 6:10 AM

Findings Summary

Severity	Count
● Critical	0
● High	4
● Medium	5
● Low	6
● Info	1
Total	16

contracts/vault/src/lib.rs

● HIGH TVL-loss guard can be disabled instantly

Description:

The ``max_loss_bps`` “TVL guard” is meant to constrain NAV loss during manager/trader operations, but it can be set to ``0`` (disabled) immediately by the manager with no timelock/2-step commit. Because Soroban transactions can batch multiple contract invocations, a compromised (or malicious) manager can disable the guard, perform a high-loss operation (e.g., a deliberately bad ``execute_trade``/``invest`` into value-losing actions), and optionally restore the previous ``max_loss_bps`` in the same transaction—bypassing the protection entirely. Exploit path: 1) Manager (or attacker with manager key) calls ``set_max_loss_bps(..., 0)``. 2) In the same transaction, call ``execute_trade``/``invest``/``invest_lp``/``unwind`` with extreme slippage or otherwise value-destructive parameters. 3) (Optional) Call ``set_max_loss_bps(..., old_value)`` to make the change less noticeable. 4) NAV guard never triggers because it was disabled at the time of the harmful call.

Affected Code:

```
pub fn set_max_loss_bps(env: Env, caller: Address, bps: u32) {
    caller.require_auth();
    if caller != get_manager(&env) {
        panic_with_error!(&env, VaultError::NotManager);
    }
    if bps > FEE_DENOMINATOR {
        panic_with_error!(&env, VaultError::InvalidAmount);
    }
    set_max_l
    ... (truncated)
```

Recommendation:

Make disabling/reducing safety parameters require a delay (similar to fee-increase timelock) or enforce a nonzero minimum in production deployments. If ``0`` must exist, gate it behind a separate “emergency” mode with explicit signaling and/or longer delay.

● HIGH Manager can whitelist arbitrary strategy contracts

Description:

The manager can replace the strategy whitelist with arbitrary contract addresses. Whitelisted strategies are then trusted for cross-contract calls that (a) influence NAV via `get\_value` and (b) can pull tokens via `approve` + `strategy.deposit(...)`. A malicious strategy added by a compromised manager can steal approved tokens and/or lie about `get\_value` to manipulate NAV-dependent mint/burn/fee logic. Exploit path: 1) Attacker compromises manager key. 2) Calls `set\_strategies` to include an attacker-controlled strategy. 3) Calls `invest`/`invest\_lp` to approve vault funds to that strategy; the strategy steals funds when invoked. 4) Optionally manipulates `get\_value` reporting to distort share price and fee minting.

Affected Code:

```
pub fn set_strategies(env: Env, caller: Address, strategies: Vec<Address>) {
    caller.require_auth();
    if caller != get_manager(&env) {
        panic_with_error!(&env, VaultError::NotManager);
    }
    ...
    set_strategies(&env, &strategies);
}

// invest():
let expiry = env.ledger().sequenc
... (truncated)
```

Recommendation:

If the protocol's security model is not "fully trusted manager", add governance constraints: timelock for `set\_strategies`, an allowlist/registry of audited strategies (e.g., Factory-controlled), and/or multisig requirements enforced on-chain. At minimum, emit detailed events and consider a withdraw... (truncated)

● MEDIUM Share price can truncate to zero and brick withdrawals

Description:

`share\_price` is computed with integer division: `nav \* PRICE\_PRECISION / total\_supply`. If `nav \* PRICE\_PRECISION < total\_supply`, this truncates to `0`. A zero share price then causes: - `withdraw`: `base\_gross = share\_amount \* share\_price / PRICE\_PRECISION` to be `0`, making `base\_net <= 0` and the withdrawal revert (`InvalidAmount`). This can lock remaining vault funds. - `deposit`: special-cases `share\_price == 0` as “bootstrap” and mints `amount` shares 1:1, which is not the economic inverse of a near-zero price and can severely misprice new deposits. This condition can arise after extreme drawdowns (NAV collapses while shares remain) and/or misconfiguration where share/base units are not aligned (e.g., share token uses unexpected decimals), making `total\_supply` large relative to `nav`. Exploit path (DoS/liveness): drive NAV low relative to outstanding shares (e.g., via strategy losses / oracle manipulation elsewhere), leaving some residual assets; users then cannot withdraw because `share\_price` becomes `0` and withdrawals revert.

Affected Code:

```
fn share_price(nav: i128, total_supply: i128) -> i128 {
    if total_supply == 0 || nav == 0 {
        PRICE_PRECISION
    } else {
        nav * PRICE_PRECISION / total_supply
    }
}

let base_gross = share_amount * share_price / PRICE_PRECISION;
...
if base_net <= 0 {
    panic_with_error!(&env,
    ... (truncated)
```

Recommendation:

Prevent `share\_price` from ever becoming `0` by using higher precision and/or enforcing a minimum (e.g., `max(1, ...)`) and revisiting the `deposit` bootstrap branch. Consider using checked/rational math with explicit rounding policy, and add an invariant test that withdrawals remain possible for an... (truncated)

contracts/strategies/soroswap_lp/src/lib.rs

● MEDIUM LP position ignored until oracle configured

Description:

`get\_value()` returns 0 when no oracle is configured, meaning the vault's NAV can temporarily ignore a non-zero LP position. If the vault invests into this strategy before `set\_oracle` is called (or if the oracle is unset/misconfigured elsewhere), users can deposit while NAV is understated and receive too many shares, then later withdraw after the oracle is set and NAV "jumps", effectively diluting existing shareholders. Exploit path: 1) Vault deposits into this strategy (LP balance becomes non-zero). 2) Oracle is not yet set for the strategy, so `get\_value()` contributes 0 to vault NAV. 3) Attacker deposits into the vault at the understated share price, receiving excess shares. 4) Manager later configures the oracle; vault NAV increases to include LP value. 5) Attacker withdraws shares for more value than paid, extracting value from prior share holders.

Affected Code:

```
let oracle = match get_oracle(&env) {  
    Some(o) => o,  
    // Without an oracle the LP units are not base-asset-denominated;  
    // returning them as NAV would inflate share price.  
    None => return 0,  
};
```

Recommendation:

Require an oracle to be configured before allowing `deposit\_liquidity` (and/or before the vault can include this strategy), or implement a safer interim valuation mechanism that cannot systematically understate NAV across user entry/exit (and ensure docs match the chosen behavior).

● LOW Reserve ordering assumption can misprice NAV

Description:

`get\_value()` assumes that `pair.get\_reserves()` returns `(reserve\_a, reserve\_b)` matching the strategy's configured `asset\_a` and `asset\_b`. Many AMM pair contracts expose reserves in a fixed internal order (e.g., token0/token1) that may not match the caller's `asset\_a/asset\_b` order. If Soroswap's pair contract behaves this way, the strategy may multiply the wrong reserve by each asset's oracle price, mispricing NAV. Exploit path (if Soroswap reserves are token0/token1 ordered): 1) Strategy is initialized with `asset\_a/asset\_b` order not matching the pair's internal ordering. 2) `get\_value()` assigns prices to the wrong reserves. 3) Vault share price becomes systematically wrong, enabling unfair deposits/withdrawals (dilution/arbitrage) against other vault users.

Affected Code:

```
let (reserve_a, reserve_b) = pair.get_reserves();
...
let pool_value_a = reserve_a.checked_mul(price_a)? / PRICE_PRECISION;
let pool_value_b = reserve_b.checked_mul(price_b)? / PRICE_PRECISION;
```

Recommendation:

Bind reserves to assets explicitly: query the pair for token addresses (e.g., token0/token1) and map reserves accordingly, or validate at initialization that the configured `asset\_a/asset\_b` ordering matches the pair's reserve ordering.

● LOW Stored strategy manager cannot use controls

Description:

`initialize` requires `manager.require\_auth()` and persists `manager`, and comments state this address is “allowed to pause / unpause”. However, `pause`, `unpause`, and `set\_oracle` ignore the stored manager and instead require the *vault's* current manager. This role mismatch can break intended separation-of-duties and can prevent the designated strategy manager from acting in emergencies (operational/liveness risk), despite having been required to authorize initialization. Impact path: 1) Strategy initialized with a distinct `manager` address intended for emergency actions. 2) Later, only the vault manager can pause/unpause/set oracle; the strategy manager cannot. 3) If vault manager is unavailable/rotated/compromised, intended emergency control procedures may fail.

Affected Code:

```
// initialize()
manager.require_auth();
...
set_manager(&env, &manager);

// set_oracle/pause/unpause
caller.require_auth();
let vault_manager: Address = env.invoke_contract(&vault, &Symbol::new(&env, "get_manager"), ().into_val(&env));
if caller != vault_manager { panic_with_error!(&env, SoroswapLp
... (truncated)
```

Recommendation:

Either (a) enforce the stored `manager` for pause/unpause/set_oracle as documented, or (b) remove the `manager` parameter/storage and update docs and initialization auth requirements to match the actual authority model.

contracts/integration_tests/src/test_factory_registry.rs

● LOW Unsafe transmute extends client lifetime unsafely

Description:

The test coerces a `FactoryClient` with a non-`static` lifetime into `FactoryClient<'static>` via `unsafe { core::mem::transmute(...) }`. This is undefined behavior if the client outlives the `Env` it internally borrows, and can cause use-after-free style bugs or flaky tests that mask real contract issues.

Affected Code:

```
let factory = FactoryClient::new(&env, &factory_id);

let factory: FactoryClient<'static> = unsafe { core::mem::transmute(factory) };
```

Recommendation:

Avoid `transmute` for lifetime extension. Prefer patterns that keep the client lifetime tied to `Env` (e.g., store the contract `Address` in `World` and construct the client on-demand, or restructure `World` so the client does not require `static`). If a `static` client is truly required, use Soro... (truncated)

● INFO mock_all_auths can miss missing require_auth

Description:

All tests run under `env.mock\_all\_auths()`, which bypasses `require\_auth` enforcement. While some negative tests use a `rogue` admin parameter, they do not cover the critical case where an unauthorized caller supplies the *real* stored admin address as the function parameter (an authorization-by-parameter failure mode). If the Factory contract accidentally omits `require\_auth` and only compares `admin` parameter equality (or worse, omits both), these integration tests may still pass and allow a production auth bypass to ship.

Affected Code:

```
// NOTE: mock_all_auths() bypasses all authorization checks in these tests.
...
env.mock_all_auths();
...
w.factory.register_vault(&w.admin, &vault_id, &manager);
...
let rogue = Address::generate(&w.env);
w.factory.register_vault(&rogue, &vault_id, &manager);
```

Recommendation:

Add at least one integration test that disables `mock\_all\_auths()` (or uses `mock\_auths` to be explicit) and asserts that calls fail when the invoker is not authorized *even if* they pass the correct stored admin address as the `admin` parameter. This specifically defends against authorization-by-pa... (truncated)

contracts/mock_blend_pool/src/lib.rs

● MEDIUM Instance config can expire and brick pool

Description:

Critical configuration (`Admin`, `Token`) is stored in instance storage but its TTL is never extended. Unlike the per-user persistent keys (which call `extend\_ttl`), the instance keys can expire/archived due to rent/TTL rules, after which `submit`/`add\_yield` will fail with `NotInitialized` when reading `DataKey::Token`/`DataKey::Admin`. If the pool still holds tokens (or has outstanding supplier state), this can permanently lock funds in the contract until state is restored (if possible in the deployment environment).

Affected Code:

```
pub fn initialize(env: Env, admin: Address, token: Address) {  
    if env.storage().instance().has(&DataKey::Admin) {  
        panic_with_error!(&env, PoolError::NotInitialized);  
    }  
    admin.require_auth();  
    env.storage().instance().set(&DataKey::Admin, &admin);  
    env.storage().instance().set  
    ... (truncated)
```

Recommendation:

Extend/bump instance storage TTL on initialize and on any stateful entrypoint (or use a shared storage helper like the vault does). Example:
`env.storage().instance().extend\_ttl(INSTANCE\_LIFETIME\_THRESHOLD, INSTANCE\_BUMP\_AMOUNT)` or per-key extension if appropriate.

● LOW Per-address supply entries enable state bloat

Description:

The contract persists a separate `DataKey::Supply(Address)` entry per supplying address. An attacker can create many new Stellar addresses and call `submit` with `REQUEST_SUPPLY` and a minimal `amount` to force unbounded persistent state growth across transactions. Even if the attacker pays the rent/fees to create/extend these entries, the contract's state will permanently grow, increasing long-term rent and operational costs and potentially impacting reliability of any environment where this mock is (mis)used beyond tests.

Affected Code:

```
enum DataKey {
    Admin,
    Token,
    // Supply(Address) creates one persistent ledger entry per depositor address.
    Supply(Address),
    TotalSupply,
}

let bal: i128 = env
    .storage()
    .persistent()
    .get(&DataKey::Supply(from.clone()))
    .unwrap_or(0);
...
env.storage().persisten
... (truncated)
```

Recommendation:

If this contract is ever used outside testing: enforce a minimum deposit, cap the number of suppliers, or avoid per-address persistent state (e.g., aggregate-only accounting or off-chain indexing). Consider shorter TTLs for ephemeral test state.

contracts/integration_tests/src/test_vault_soroswap_lp.rs

● LOW Tests bypass authorization checks, hiding missing require_auth

Description:

The test harness enables `env.mock_all_auths_allowing_non_root_auth()`, which auto-approves all authorization at every call depth. This can cause integration tests to pass even if production contract entrypoints are missing or incorrectly scoped `require_auth` checks (especially for cross-contract call stacks), reducing the chance of catching real authorization bugs before deployment.

Affected Code:

```
env.mock_all_auths_allowing_non_root_auth();
```

Recommendation:

Where tests are meant to validate authorization, avoid blanket auth mocking. Prefer `mock_all_auths()` (or explicit auth setups) and assert `require_auth` coverage by inspecting recorded auths / using negative tests that should fail without proper signatures. Reserve `allowing_non_root_auth` only fo... (truncated)

contracts/factory/src/storage.rs

● LOW Persistent registry entries can silently expire

Description:

The factory stores per-vault registry data (`VaultByIndex`, `VaultPosition`, `IsRegistered`) in ****persistent**** storage and only extends their TTL when a specific key is accessed via `bump\_persistent`. If some indexes are not accessed for longer than their TTL, those entries can expire while `VaultCount` (kept in instance storage and bumped frequently) continues to indicate they exist. This can permanently corrupt the registry's index invariants (e.g., `vault\_count` points past missing `VaultByIndex(i)` entries), causing panics/DoS for enumeration (`get\_vaults`) and potentially making some vaults non-removable/undiscoverable through the factory once their reverse-index/membership keys expire. While this is primarily a liveness/integrity issue (not direct fund loss), it is Soroban-specific and can break core registry functionality over time if UIs/users only ever query the first pages and older/high-index entries are never "touched."

Affected Code:

```
pub const PERSISTENT_BUMP_AMOUNT: u32 = 5_256_000;
pub const PERSISTENT_LIFETIME_THRESHOLD: u32 = 2_628_000;

fn bump_persistent(env: &Env, key: &DataKey) {
    env.storage()
        .persistent()
        .extend_ttl(key, PERSISTENT_LIFETIME_THRESHOLD, PERSISTENT_BUMP_AMOUNT);
}

pub fn get_vault_by
... (truncated)
```

Recommendation:

Ensure registry entries cannot silently expire relative to `VaultCount`. Options include: (a) set much longer persistent TTLs (multi-year) and/or bump them more aggressively; (b) add an admin/public "touch" function to bump TTL for a specified index range; (c) consider keeping critical index metadat... (truncated)

contracts/strategies/blend/src/storage.rs

● HIGH State expiry enables strategy reinitialization takeover

Description:

The strategy stores its critical ownership/configuration (Vault/Asset/Protocol/Manager) exclusively in **instance storage** and considers the contract initialized only if `DataKey::Vault` exists. Instance storage is TTL-bound; if the instance entries expire (e.g., no one calls the strategy for `> INSTANCE_BUMP_AMOUNT` ledgers), `is_initialized()` will return false and a new caller can re-run initialization logic in `lib.rs` to set themselves/their vault as the controlling addresses. Exploitation path (high impact): After users/vault have supplied funds to Blend (position owned by the strategy contract address), if the strategy's instance storage expires, an attacker can call the strategy's `initialize` again (passing a vault they control, and satisfying auth checks against their own vault/manager), then call `withdraw(..., from=<attacker vault>, to=<attacker>)` to drain the still-existing Blend position from the strategy contract to themselves.

Affected Code:

```
/// Ledgers added to the instance entry TTL on every entry-point call.
/// 518 400 ledgers ≈ 30 days.
pub const INSTANCE_BUMP_AMOUNT: u32 = 518_400;

/// Trigger a bump when the remaining instance TTL drops below this value.
/// 259 200 ledgers ≈ 15 days.
pub const INSTANCE_LIFETIME_THRESHOLD: u32 =
... (truncated)
```

Recommendation:

Avoid relying on expirable instance storage as the sole initialization/ownership anchor for a contract that can custody external positions.

Mitigations:

- Add an additional initialization guard that cannot be reset by TTL expiry (e.g., a persistent key with a long TTL plus an explicit off-chain kee... (truncated)

contracts/factory/src/lib.rs

● HIGH Instance TTL expiry enables factory reinitialization takeover

Description:

The factory's initialization guard relies on `has_admin(&env)` (instance storage). Because instance entries are TTL-bound and bumped only on calls, a sufficiently idle factory can have its instance state (Admin/VaultCount) expire. After expiry, `has_admin(&env)` will return false and any attacker can call `__constructor` with themselves as `admin` (they can satisfy `admin.require_auth()` with their own signature), thereby taking over as factory admin. This can also corrupt registry enumeration because `VaultCount` is instance-stored and would reset to 0 while per-vault keys are persistent, making existing registered vaults potentially undiscoverable via `get_vaults` and breaking `remove_vault` due to `count - 1` underflow/invalid last index reads. Exploit path: 1) Wait until the factory has no calls for long enough that its instance TTL expires. 2) Call `Factory::__constructor(env, attacker_address)` and authorize as `attacker_address`. 3) Attacker becomes admin and can register attacker-controlled "vaults"/manipulate the registry; UIs/indexers relying on the factory can be tricked into presenting malicious vaults as canonical.

Affected Code:

```
pub fn __constructor(env: Env, admin: Address) {
    if has_admin(&env) {
        panic_with_error!(&env, FactoryError::AlreadyInitialized);
    }
    admin.require_auth();
    env.storage()
        .instance()
        .extend_ttl(INSTANCE_LIFETIME_THRESHOLD, INSTANCE_BUMP_AMOUNT);

    set_admin(&
... (truncated)
```

Recommendation:

Do not rely on short-lived instance storage as the sole initialization/admin anchor. Store `Admin` (and preferably an explicit `Initialized` flag and `VaultCount`) in persistent storage with a long TTL (or periodically bump TTL via a dedicated keeper), and ensure reinitialization is impossible even ... (truncated)

contracts/trade_guards/phoenix/src/lib.rs

● MEDIUM Swap quote path can diverge from operations

Description:

`validate\_swap` computes `quoted\_out` by converting the provided `operations` into a flat `path` that assumes hop-to-hop continuity, but it never enforces that continuity. Specifically, it builds `path` as `[operations[0].offer\_asset, operations[0].ask\_asset, operations[1].ask\_asset, ...]`, ignoring `operations[i>0].offer\_asset` entirely. If a caller supplies an `operations` list where `operations[i].offer\_asset != operations[i-1].ask\_asset`, the guard's quoted path may not correspond to the actual operations that will be executed on the Phoenix router, so the slippage check can be performed against a different route than the one being traded. Exploitation path (if the vault/strategy uses `validate\_swap` for execution): a malicious/compromised trader supplies a multi-hop `operations` list crafted to (a) pass whitelist checks, (b) produce a favorable `quoted\_out` for the derived `path`, and (c) execute a different (worse) sequence of swaps in the router, potentially bypassing the intended slippage constraint and causing larger-than-allowed vault losses (or at minimum causing unexpected failures/DoS due to inconsistent operations).

Affected Code:

```
for op in operations.iter() {
    Self::assert_whitelisted(&op.offer_asset, &whitelist, &env);
    Self::assert_whitelisted(&op.ask_asset, &whitelist, &env);
}

// Build a flat path [offer_0, ask_0, ask_1, ...] from the operations so
// we can call the router's generic quote_exact_in interface.
let mu
... (truncated)
```

Recommendation:

In `validate\_swap`, enforce hop continuity: for all `i>0`, require `operations[i].offer\_asset == operations[i-1].ask\_asset`. Additionally, consider deriving the quote input from the exact structure passed to the router (or remove `validate\_swap` entirely and standardize on `validate_swap_exact_in(pa... (truncated)

contracts/strategies/phoenix_lp/src/lib.rs

● MEDIUM Strategy can report zero value for active LP position

Description:

`get\_value` returns `0` when no oracle is configured (and also when `total\_shares == 0` or pool reserves are both zero), even if this strategy is actually holding Phoenix LP share tokens. The vault relies on `strategy.get\_value(vault) == 0` as the safety condition that allows a manager to remove a strategy from the whitelist without having to unwind it first. Because `vault.invest\_lp` only checks that `strategy\_has\_oracle()` is true (a boolean view) and this strategy has no on-chain linkage enforcing oracle presence at initialization, any misconfiguration (or a manager intentionally delaying oracle configuration before investing) can lead to `get\_value` returning `0` while funds are still parked in the pool. In that state, `Vault::set\_strategies` will allow removing the strategy, silently abandoning an active position and understating NAV. Exploit path (manager-controlled but bypasses an intended guard): (1) set an oracle that makes `has\_oracle()` true or invest before oracle is configured depending on vault integration, (2) invest into the Phoenix pool so the strategy holds LP shares, (3) ensure `get\_value` returns `0` (e.g., oracle unset/mis-set so `get\_oracle` is `None`, or pool reports zero reserves/total supply), (4) call `Vault::set\_strategies` to remove the strategy; the vault's removal guard passes because `value == 0`, leaving assets stranded and NAV understated, enabling share-price manipulation/dilution and potentially permanent loss if the vault no longer interacts with the removed strategy.

Affected Code:

```
pub fn get_value(env: Env, _vault: Address) -> i128 {
    ...
    let shares = token::Client::new(&env, &share_token).balance(&strategy);
    if shares == 0 {
        return 0;
    }

    let oracle = match get_oracle(&env) {
        Some(o) => o,
        None => return 0,
    };

    let (reserve_a
    ... (truncated)
```

Recommendation:

Make `get\_value` return a non-zero indicator whenever the strategy holds LP shares, even if pricing is unavailable. For example: return the live LP share-token balance (or another monotonic position metric) when the oracle is missing, and have the vault removal check use a dedicated `get\_position\_un...` (truncated)